

FATEC VOTORANTIM

Curso Superior de Tecnologia em Ciência de Dados para Negócios

Ana Elisa Rubinato Cavalheiro

Arthur Nunes dos Santos

Bruno Felipe de Souza Araujo

Caio Cesar Corr 

Lucas Camelo

Nicole Fava Pires

DEPLOY GUIDE

Intelig ncia de Risco em Comodato de Chopp

Projeto Integrador V

Controle de Versão

Registro histórico de todas as revisões deste documento, com data, responsável e descrição da alteração.

Tabela 1 - Controle de Versão

Versão	Data	Autor	Descrição
1.0	16/04/2026	Equipe PI5	Criação inicial do documento
1.1	16/05/2026	Equipe PI5	Revisão do site e simulação do modelo preditivo
1.2	01/06/2026	Equipe PI5	Ajustes para deploy em PythonAnywhere
1.3	25/05/2026	Equipe PI5	Ajustes pós revisão

Fonte: Autoria própria

DEPLOY GUIDE

Controle de Versão	1
1. Objetivo	3
2. Escopo.....	3
3. Definições e Siglas.....	3
4. Visão Geral da Solução	4
4.1 Contexto de Negócio	4
4.2 Tipo do Modelo	4
4.3 Regra de Negócio do Target.....	4
4.4 Fluxo da Solução	4
5. Arquitetura da Solução.....	5
5.1 Componentes	5
5.2 Separação de Datasets	5
5.3 Pipeline do Modelo (sklearn)	5
6. Estrutura do Projeto	6
7. Pré-requisitos	6
7.1 requirements.txt	6
7.2 Bibliotecas de ciência de dados	7
8. Deploy no PythonAnywhere	7
8.1 Criar conta e acessar o painel	7
8.2 Upload dos arquivos	8
8.3 Instalar as dependências	8
8.4 Criar a Web App.....	8
8.5 Configurar caminhos no app.py	9
8.6 Iniciar o site	9
9. Variáveis de Ambiente	9
10. Versionamento do Modelo	10
10.1 Convenção de nomes dos PKLs	10
10.2 Metadados armazenados no PKL	10
10.3 Como trocar o modelo em produção	11
11. Atualização do Modelo.....	11
11.1 Quando retreinar.....	11
11.2 Ciclo de retreinamento	11
12. Calibração do Threshold	12
13. Testes	12
13.1 Verificar saúde da aplicação.....	12
13.2 Testar predição via formulário	13
14. Monitoramento	13
15. Riscos e Mitigações	14
16. Próximos Passos Recomendados	14
17. Responsáveis	15
18. Aprovação	15

1. Objetivo

Este guia descreve o processo de implantação (deploy), atualização e manutenção do modelo preditivo de risco de clientes da distribuidora Chopp & Cia, desenvolvido no âmbito do Projeto Integrador V da FATEC Votorantim. A plataforma escolhida para hospedagem é o PythonAnywhere, um serviço de nuvem que permite executar aplicações Python/Flask diretamente no navegador, sem necessidade de servidor próprio, Docker ou configurações complexas de infraestrutura.

2. Escopo

Este documento cobre todas as etapas necessárias para colocar e manter o sistema em funcionamento:

- Upload e organização dos arquivos no PythonAnywhere
- Configuração da aplicação Flask como Web App
- Atualização do modelo treinado (arquivo .pkl) e do dataset
- Procedimento de reinicialização do site após qualquer mudança
- Testes básicos de funcionamento após o deploy
- Versionamento do modelo e ciclo de retreinamento

3. Definições e Siglas

Glossário dos termos técnicos e siglas utilizados ao longo deste documento.

Tabela 2 - Definições e Siglas

Termo	Descrição
Flask	Framework web em Python utilizado na API do sistema
PythonAnywhere	Plataforma de hospedagem na nuvem para aplicações Python, acessível em pythonanywhere.com
WSGI	Interface padrão entre servidor web e aplicação Python — configurada automaticamente pelo PythonAnywhere
PKL	Arquivo serializado em Python (pickle) contendo o pipeline do modelo treinado
Pipeline	Sequência encadeada de pré-processamento + classificador (<code>sklearn.Pipeline</code>)
df_ml_completo	Dataset CSV com todos os clientes e suas features calculadas — usado para busca por ID
ID_PESSOA	Identificador único do cliente no sistema ERP
AUC-ROC	Métrica de qualidade do modelo: quanto mais próximo de 1.0, melhor
Threshold	Limiar de probabilidade para classificar como Alto Risco (padrão: 0.50)
Reload	Ação de reiniciar o servidor web no PythonAnywhere para aplicar mudanças

Fonte: Autoria própria

4. Visão Geral da Solução

Esta seção resume o contexto de negócio, o tipo de modelo utilizado e como os dados fluem desde a entrada do usuário até a resposta da predição.

4.1 Contexto de Negócio

A Chopp & Cia é uma distribuidora de chopp localizada em Ponta Porã (MS), na fronteira Brasil–Paraguai. O sistema avalia dois tipos de risco simultâneos para cada cliente:

- Risco Financeiro: cliente atrasa ou não paga parcelas de vendas de chopp
- Risco de Ativo (Comodato): cliente não devolve equipamentos emprestados (chopeiras)

4.2 Tipo do Modelo

Classificação Binária Supervisionada com target ALTO_RISCO (0 = Bom Pagador, 1 = Alto Risco). O algoritmo utilizado é Random Forest.

4.3 Regra de Negócio do Target

A definição de risco segue a seguinte lógica, aplicada somente a clientes com histórico mínimo (mais de 2 compras):

```
ALTO_RISCO = 1 se TAXA_ATRASO_PAGAMENTO > 20% OU TAXA_ATRASO_COMODATO > 20%  
ALTO_RISCO = 0 caso contrário
```

```
Filtro de veteranos: apenas clientes com FREQUENCIA_COMPRAS > 2 entram no treino
```

4.4 Fluxo da Solução

O caminho percorrido desde a solicitação do usuário até a exibição do resultado no site:

- Usuário informa o ID_PESSOA ou preenche os dados manualmente no formulário do site
- A API Flask busca as features do cliente em df_ml_completo (lookup por ID)
- O pipeline de pré-processamento é aplicado: StandardScaler (numéricas) + OneHotEncoder (categóricas)
- O modelo calcula a probabilidade de risco via predict_proba
- A API classifica: BAIXO (< 35%) · MÉDIO (35–65%) · ALTO (> 65%)
- O resultado é exibido no site com probabilidade, nível de risco e dados do modelo

5. Arquitetura da Solução

Visão técnica dos componentes que compõem o sistema e como eles se integram na plataforma PythonAnywhere.

5.1 Componentes

Tabela 3 - Componentes

Componente	Tecnologia	Função
Notebook de Treino	Python / Jupyter	Feature Engineering, treinamento, validação e exportação do PKL
Modelo (PKL)	scikit-learn + pickle	Pipeline serializado com pré-processamento e classificador
Dataset de Lookup	CSV / Pandas	df_ml_completo.csv com features de todos os clientes
API Web	Flask 3.0	Endpoints /predict e /buscar_cliente — servidos pelo PythonAnywhere
Interface Web	HTML + JavaScript	Formulário de entrada e exibição dos resultados para o usuário
Servidor	PythonAnywhere	Hospedagem gratuita com suporte nativo a Flask/WSGI

Fonte: Autoria própria

5.2 Separação de Datasets

É fundamental não confundir os dois datasets do projeto. Eles têm funções distintas:

`df_ml_completo.csv` → Tabela de usuários em produção.
Contém `ID_PESSOA`, `NOME_CLIENTE` e todas as features. Usados exclusivamente para lookup no site.

`df_treino` (interno ao notebook) → Dataset de modelagem.
Subconjunto de veteranos, sem colunas de identidade. Usado apenas durante o treinamento.

Nunca misturar os dois em produção.

5.3 Pipeline do Modelo (sklearn)

O modelo é encapsulado em um Pipeline do scikit-learn com dois estágios: pré-processamento e classificação. Esse pipeline é serializado no arquivo `.pkl` junto com os metadados do modelo.

- `prep`: `ColumnTransformer` com `StandardScaler` (features numéricas) + `OneHotEncoder` (features categóricas)
- `clf`: `RandomForestClassifier`

6. Estrutura do Projeto

Organização esperada dos arquivos e pastas no PythonAnywhere. Todos os caminhos são relativos ao diretório raiz do usuário (/home/seu_usuario/).

```
ProjetoIntegrador/
├── app.py                    # Aplicação Flask principal
├── requirements.txt          # Dependências do projeto
├── templates/
│   └── index.html            # Interface web do sistema
├── modelo/
│   └── modelo_chopp_v1.0.0_random_forest.pkl  # Modelo treinado
└── data/
    └── df_ml_completo.csv     # Dataset de lookup de clientes
```

O arquivo `app.py` já está configurado para buscar o modelo e o dataset nos caminhos acima. Para mudar os caminhos, edite as variáveis `MODEL_PATH` e `DATASET_PATH` no início do `app.py`, ou use variáveis de ambiente do PythonAnywhere (recomendado).

7. Pré-requisitos

O PythonAnywhere já fornece Python 3.10+ e pip nativamente, sem necessidade de instalação adicional. As bibliotecas abaixo precisam ser instaladas via console antes de iniciar a aplicação.

7.1 requirements.txt

O arquivo `requirements.txt` do projeto contém apenas as dependências necessárias para o servidor web. As bibliotecas de ciência de dados (pandas, scikit-learn etc.) precisam ser instaladas separadamente, pois são usadas tanto pelo notebook quanto pela API.

```
# requirements.txt - dependências do servidor web
Flask==3.0.0
Werkzeug==3.0.1
gunicorn==21.2.0
```

7.2 Bibliotecas de ciência de dados

Além das dependências acima, a aplicação também depende das bibliotecas a seguir. No console do PythonAnywhere, instale com:

```
pip install --user pandas scikit-learn numpy
```

Tabela 4 - Bibliotecas

Biblioteca	Versão Mínima	Finalidade
pandas	2.0+	Leitura do CSV e manipulação de DataFrames
scikit-learn	1.3+	Carregamento e execução do pipeline treinado
numpy	1.24+	Operações numéricas internas do pipeline
Flask	3.0+	Framework da aplicação web
Werkzeug	3.0+	Utilitários internos do Flask

Fonte: Autoria própria

A versão do scikit-learn instalada no PythonAnywhere deve ser a mesma utilizada durante o treinamento do modelo. Diferenças de versão podem impedir o carregamento do PKL. Para verificar: `python3 -c "import sklearn; print(sklearn.__version__)"`

8. Deploy no PythonAnywhere

Esta seção descreve o passo a passo completo para publicar o site no PythonAnywhere pela primeira vez. O processo leva cerca de 10 a 15 minutos e não requer nenhuma ferramenta adicional além do navegador.

8.1 Criar conta e acessar o painel

Acesse pythonanywhere.com e crie uma conta gratuita (plano Beginner). Após o login, o painel principal exibe abas para gerenciar arquivos, consoles, aplicações web e tarefas agendadas.

8.2 Upload dos arquivos

Todos os arquivos do projeto devem ser enviados para o servidor do PythonAnywhere. O caminho de destino padrão é `/home/seu_usuario/ProjetoIntegrador/`.

- Acesse a aba Files no painel do PythonAnywhere
- Navegue até `/home/seu_usuario/` e crie a pasta `ProjetoIntegrador`
- Dentro dela, crie as subpastas `templates/`, `modelo/` e `data/`
- Envie `app.py` e `requirements.txt` para a raiz (`ProjetoIntegrador/`)
- Envie `index.html` para `templates/`
- Envie o arquivo `.pkl` para `modelo/`
- Envie `df_ml_completo.csv` para `data/`

Para arquivos grandes (CSV ou PKL acima de 100 MB), o upload via interface web pode demorar. Alternativa: usar o console do PythonAnywhere com `wget` ou `git clone` a partir do GitHub.

8.3 Instalar as dependências

Abra um console Bash no PythonAnywhere (aba Consoles → Bash) e execute os comandos abaixo:

```
cd ~/ProjetoIntegrador
pip install --user -r requirements.txt
pip install --user pandas scikit-learn numpy
```

8.4 Criar a Web App

Com os arquivos no lugar e as dependências instaladas, configure a aplicação web para que o PythonAnywhere passe as requisições para o Flask:

- Acesse a aba Web no painel e clique em Add a new web app
- Escolha Manual configuration (não use os atalhos de framework)
- Selecione Python 3.10 (ou a versão disponível mais próxima de 3.11)
- Na seção Code, defina o Source code como `/home/seu_usuario/ProjetoIntegrador`
- Na seção WSGI configuration file, clique no link do arquivo `wsgi.py` para editá-lo

Substitua TODO o conteúdo do arquivo `wsgi.py` pelo código abaixo:

```
import sys

project_home = '/home/seu_usuario/ProjetoIntegrador/Site'

if project_home not in sys.path:
    sys.path = [project_home] + sys.path

from app import app as application
```

8.5 Configurar caminhos no app.py

O app.py usa caminhos relativos por padrão, construídos a partir do diretório do próprio arquivo. Se a estrutura de pastas estiver correta (ver Seção 6), nenhuma edição é necessária. Caso precise ajustar, localize as variáveis MODEL_PATH e DATASET_PATH no início do arquivo e corrija os caminhos absolutos:

```
MODEL_PATH =  
'/home/seu_usuario/ProjetoIntegrador/modelo/modelo_chopp_v1.0.0_random_forest.pkl'  
DATASET_PATH =  
'/home/seu_usuario/ProjetoIntegrador/data/df_ml_completo.csv'
```

8.6 Iniciar o site

Após configurar o WSGI e os caminhos, clique no botão verde Reload na aba Web. O PythonAnywhere irá iniciar a aplicação. O endereço público do site será exibido no topo da página, no formato:

```
https://seu_usuario.pythonanywhere.com
```

Acesse o endereço no navegador e verifique se o site carrega corretamente. Para confirmar que o modelo e o dataset foram carregados, acesse o endpoint de saúde:

```
https://seu_usuario.pythonanywhere.com/health
```

9. Variáveis de Ambiente

Variáveis de ambiente permitem alterar configurações do sistema sem precisar editar o código-fonte. No PythonAnywhere, elas são configuradas na aba Web → Virtualenv / Environment variables. As variáveis abaixo são suportadas pelo app.py:

Tabela 5 - Variáveis de Ambiente

Variável	Descrição	Exemplo de valor
MODEL_PATH	Caminho absoluto do arquivo .pkl do modelo	/home/user/ProjetoIntegrador/modelo/modelo_chopp_v1.0.0_random_forest.pkl
DATASET_PATH	Caminho do arquivo df_ml_completo.csv	/home/user/ProjetoIntegrador/data/df_ml_completo.csv
THRESHOLD	Limiar de classificação de risco (0 a 1)	0.50

Fonte: Autoria própria

Se as variáveis não forem definidas, o app.py utilizará os caminhos relativos padrão descritos na Seção 6.

10. Versionamento do Modelo

Cada versão do modelo treinado é salva como um arquivo .pkl com nome padronizado. Manter versões anteriores permite reverter para um modelo mais estável caso o novo apresente problemas.

10.1 Convenção de nomes dos PKLs

O nome do arquivo segue o padrão abaixo, facilitando a identificação da versão e do algoritmo:

```
modelo_chopp_v{versao}_{algoritmo}.pkl
```

Exemplos:

```
modelo_chopp_v1.0.0_random_forest.pkl ← versão atual
```

```
modelo_chopp_v1.1.0_random_forest.pkl ← próximo ajuste de parâmetros
```

```
modelo_chopp_v2.0.0_xgboost.pkl      ← mudança de algoritmo
```

10.2 Metadados armazenados no PKL

Além do pipeline treinado, cada arquivo .pkl contém um dicionário com informações sobre o modelo, acessíveis em produção via `modelo.get('campo')`:

Tabela 6 - Descrição de metadados

Campo	Tipo	Descrição
pipeline	sklearn.Pipeline	ColumnTransformer + Classificador serializado
nome_modelo	str	Ex: Random Forest (Tuned)
versao	str	Semântico: MAJOR.MINOR. PATCH
features_num	list[str]	9 features numéricas usadas no treino
features_cat	list[str]	3 features categóricas (PERFIL, CIDADE, PAGAMENTO)
metricas_teste	dict	AUC, F1, Recall, Precisão, Accuracy no conjunto de teste
data_treinamento	str	Data/hora do treinamento no formato ISO 8601

Fonte: Autoria própria

10.3 Como trocar o modelo em produção

Sempre que um novo modelo for treinado e validado no notebook, siga os passos abaixo para colocá-lo em produção de forma segura:

- Treine o novo modelo no notebook e execute `save_model_version()` para gerar o novo PKL
- Compare as métricas do novo modelo com o atual usando `compare_models()` no notebook
- Faça o upload do novo PKL para a pasta `modelo/` no PythonAnywhere (mantendo o anterior)
- Atualize `MODEL_PATH` no `app.py` ou na variável de ambiente para apontar para o novo arquivo
- Clique em Reload na aba Web para reiniciar a aplicação com o novo modelo
- Acesse `/health` para confirmar que o novo modelo foi carregado corretamente
- Registre a troca na tabela de Controle de Versão deste documento

11. Atualização do Modelo

O modelo deve ser retreinado periodicamente para refletir mudanças no comportamento dos clientes ao longo do tempo. Esta seção descreve quando retreinar e quais passos seguir.

11.1 Quando retreinar

Três situações indicam que o modelo precisa ser atualizado:

- Retreinamento trimestral de rotina, com dados dos últimos 6 meses
- Queda de AUC-ROC superior a 5% em relação à linha base do modelo atual
- Mudança significativa no perfil de clientes (expansão geográfica, novos produtos, sazonalidade)

11.2 Ciclo de retreinamento

Passo a passo para executar um ciclo completo de atualização do modelo:

- Exportar os CSVs atualizados do ERP (`score_vendas`, `score_financeiro`, `score_comodato`)
- Executar o notebook `Chopp_Risco_ML.ipynb` do zero, passando por todas as fases
- Comparar as métricas do novo modelo com o modelo atual em produção
- Se $AUC_{novo} \geq AUC_{atual} - 0.02$, promover o novo modelo (ver Seção 10.3)
- Exportar o novo `df_ml_completo.csv` e substituir o arquivo em `data/` no PythonAnywhere
- Arquivar o PKL anterior em uma subpasta `modelo/archive/` (nunca deletar)
- Incrementar a versão: MINOR para ajuste de parâmetros, MAJOR para novo algoritmo

12. Calibração do Threshold

O threshold padrão de 0.50 define o ponto de corte entre Bom Pagador e Alto Risco. Ele pode ser ajustado conforme a prioridade do negócio sem necessidade de retreinar o modelo — basta alterar a variável THRESHOLD.

Tabela 7 - Parâmetros

Cenário	Threshold	Efeito prático
Padrão (balanceado)	0.50	Equilíbrio entre Recall e Precisão
Priorizar detecção de risco	0.35–0.40	Recall ↑, Precisão ↓ — menos falsos negativos (mais conservador)
Priorizar precisão	0.60–0.65	Precisão ↑, Recall ↓ — menos alarmes falsos

Fonte: Autoria própria

Independentemente do threshold, os três níveis de risco são sempre calculados da seguinte forma:

```
Probabilidade < 35%           → BAIXO (Bom Pagador — aprovar)
35% ≤ Probabilidade < 65%    → MÉDIO (Bom Pagador — monitorar)
Probabilidade ≥ 65%          → ALTO (Alto Risco — não aprovar sem
garantias)
```

13. Testes

Após qualquer deploy ou atualização, execute os testes abaixo para confirmar que o sistema está funcionando corretamente antes de liberar para os usuários.

13.1 Verificar saúde da aplicação

Acesse o endpoint /health no navegador ou via curl. A resposta deve indicar que o modelo e o dataset estão carregados:

```
GET https://seu_usuario.pythonanywhere.com/health
```

Resposta esperada:

```
{ "status": "ok", "modelo_carregado": true, "dataset_carregado":
true, "n_clientes": 1234 }
```

13.2 Testar predição via formulário

Abra o site no navegador e realize os seguintes testes manuais para validar os três cenários de risco:

Tabela 8 - Detalhes da Predição

Cenário	Características do cliente	Resultado esperado
Bom Pagador	Alta frequência de compras, zero atrasos, Pessoa Jurídica, PIX	BAIXO / Bom Pagador (0)
Risco Moderado	Frequência média, atrasos leves (< 20%), Pessoa Física, Boletão	MÉDIO / Bom Pagador (0)
Alto Risco	Baixa frequência, atrasos severos (> 50 dias), Pessoa Física, Outras cidades	ALTO / Alto Risco (1)
ID inexistente	ID_PESSOA que não existe no dataset	Mensagem de ID não encontrado

Fonte: Autoria própria

14. Monitoramento

Monitorar o sistema em produção permite identificar problemas de qualidade do modelo e da aplicação antes que afetem as decisões de negócio. As verificações abaixo podem ser feitas manualmente ou com auxílio de planilhas de controle.

Tabela 9 - Monitoramento

Métrica	Frequência	Alerta se...
Taxa de Alto Risco nas predições	Diária	Acima de 50% ou abaixo de 5% — possível data drift
Distribuição das probabilidades	Semanal	Pico no centro (0.40–0.60) indica modelo incerto
Site respondendo corretamente	Contínuo	Site fora do ar ou retornando erros 5xx
AUC-ROC (reavaliação no notebook)	Trimestral	Queda superior a 5% em relação à linha base

Fonte: Autoria própria

Os logs de acesso da aplicação ficam disponíveis na aba Web do PythonAnywhere, nos arquivos error.log e access.log, úteis para diagnosticar erros em produção.

15. Riscos e Mitigações

Lista dos principais riscos identificados para o sistema em produção, com suas respectivas estratégias de mitigação.

Tabela 10 - Riscos e Mitigações

Risco	Probabilidade	Impacto	Mitigação
Versão do scikit-learn diferente entre treino e produção	Média	Alto	Fixar versão exata no requirements.txt e verificar no PythonAnywhere
Data drift: perfil dos clientes muda com o tempo	Alta	Alto	Monitoramento mensal das features; retreinamento trimestral
df_ml_completo desatualizado em produção	Média	Alto	Regenerar e re-exportar o CSV após cada ciclo de retreinamento
Modelo não cobre clientes novos (sem histórico)	Alta	Médio	Para FREQUENCIA_COMPRAS ≤ 2 , aplicar regra manual baseada em cadastro
Erro ao carregar PKL após atualização do modelo	Baixa	Alto	Manter PKL anterior disponível e testar /health imediatamente após o Reload

Fonte: Autoria própria

16. Próximos Passos Recomendados

Melhorias identificadas durante o desenvolvimento que podem ser implementadas em versões futuras do sistema, listadas por ordem de prioridade e impacto no negócio.

- Temporal Split: retreinar com dados de 2023–2024 e validar em 2025 (validação prospectiva realista)
- Cold-start: modelo alternativo para clientes novos sem histórico, baseado em PERFIL + CIDADE + PAGAMENTO
- Predição em lote: adicionar endpoint POST /predict/batch para processar múltiplos clientes de uma vez
- SHAP values: exibir na interface quais features mais contribuíram para cada predição (explicabilidade)
- Autenticação básica: proteger o acesso ao site com login simples antes de expor para usuários externos

17. Responsáveis

Integrantes da equipe responsáveis pelo desenvolvimento, implementação e manutenção do sistema.

Tabela 11 - Responsáveis

Nome	Função
Ana Elisa	Engenharia de Features / EDA
Arthur Nunes	Modelagem
Bruno Araujo	Pipeline e Versionamento
Caio Corr�	API / Deploy
Lucas Camelo	Monitoramento / Avalia�
Nicole Fava	Documenta�� / Testes

Fonte: Autoria pr pria

18. Aprova  

Espa o reservado para assinatura dos respons veis pela valida  o e aprova  o formal deste documento antes da publica  o em produ  o.

Nome	Cargo	Data	Assinatura
Arthur Nunes dos Santos	CEO	29/05/2026	



Documento assinado digitalmente

ARTHUR NUNES DOS SANTOS

Data: 28/05/2026 22:02:53-0300

Verifique em <https://validar.it.gov.br>